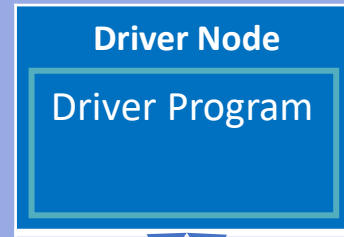
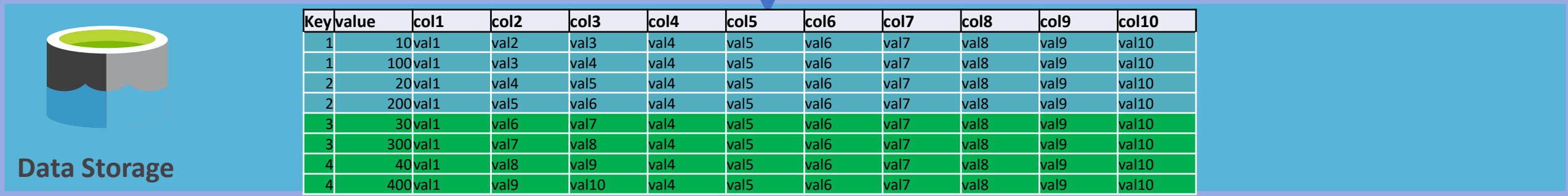
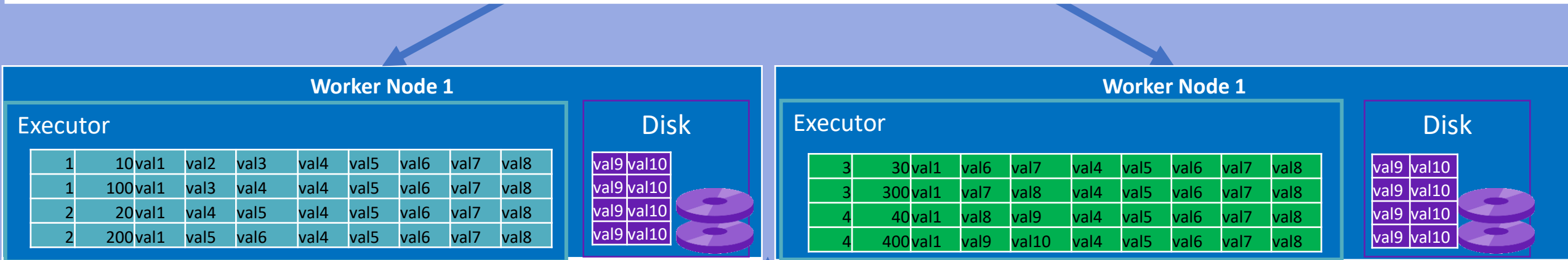


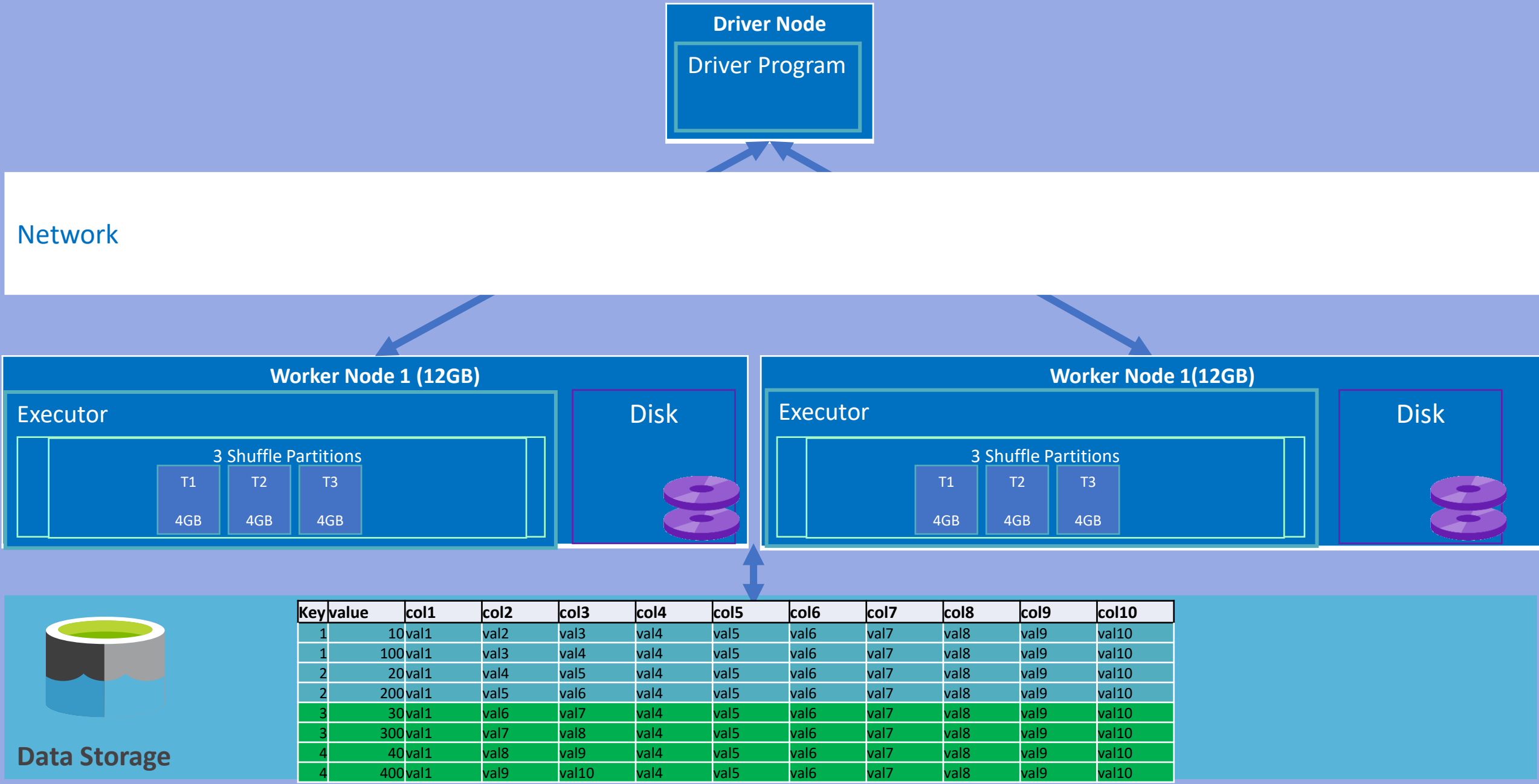
Performance Tuning – Data Spilling Issue



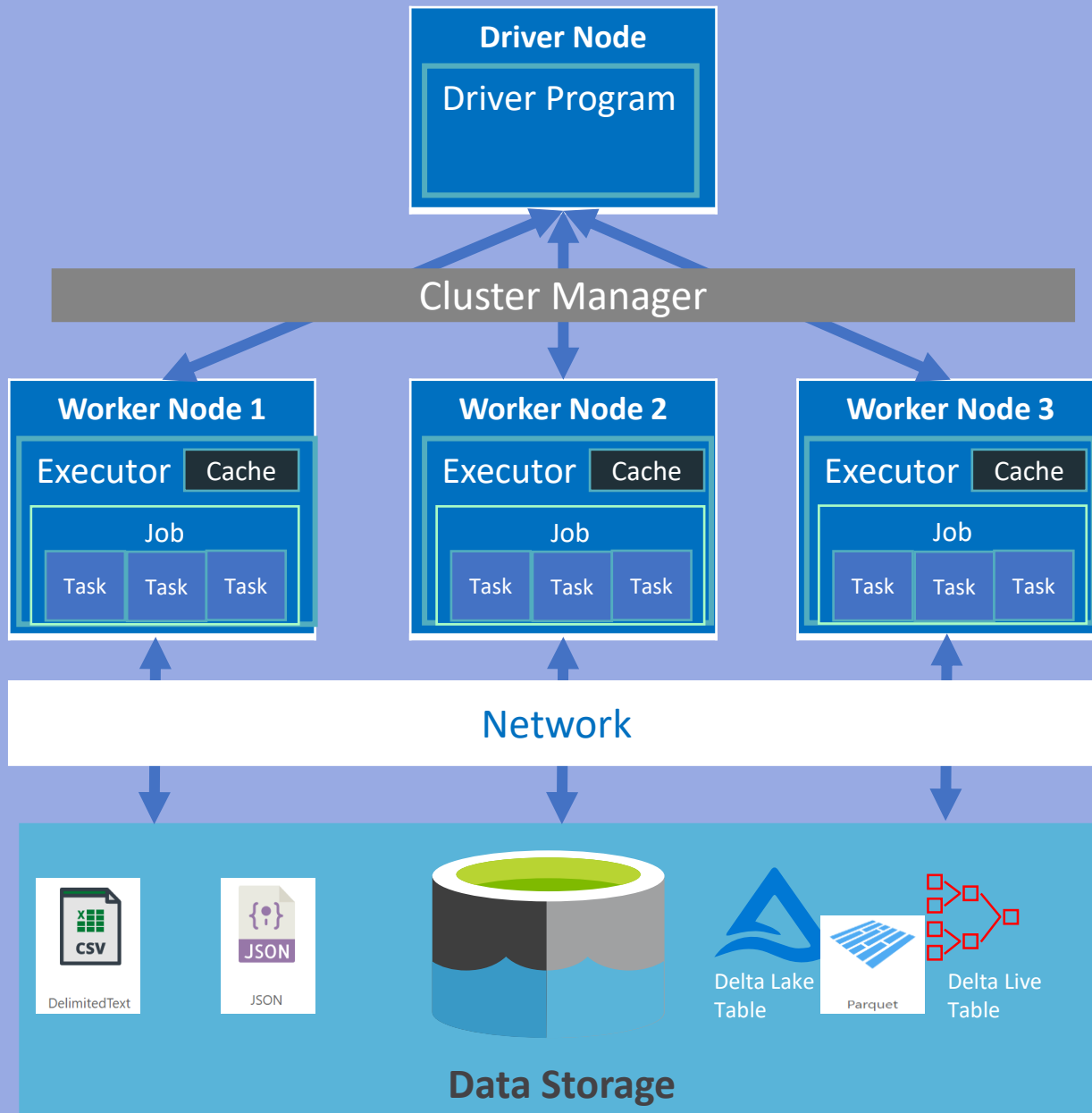
Network



Performance Tuning – Data Spilling Issue



Performance Tuning – Data Spilling(Writing to Disk) Issue



Spark SQL Shuffle Partition – Number of Partitions for data distribution(Default 200)

Task Size – Amount of Data Processed by Task (Default 128 MB) , Shares the Memory allocated In a Worker Node

Many Shuffle Partitions -> Too Many Small Tasks

Less Shuffle Partitions -> Uneven Data Distribution, Long running Tasks

ADAPTIVE QUERY EXECUTION(AQE) has a feature **autoOptimizeShuffle(AOS)** and it can be set using property set `spark.sql.shuffle.partitions=auto`

High Compression ratio on tables cause AOS to choose correct shuffle partition

Compare Total size of file read Vs Data Size After Shuffle(Exchange). If After shuffle size is drastically larger than Total size, then there is high Compressions ratio

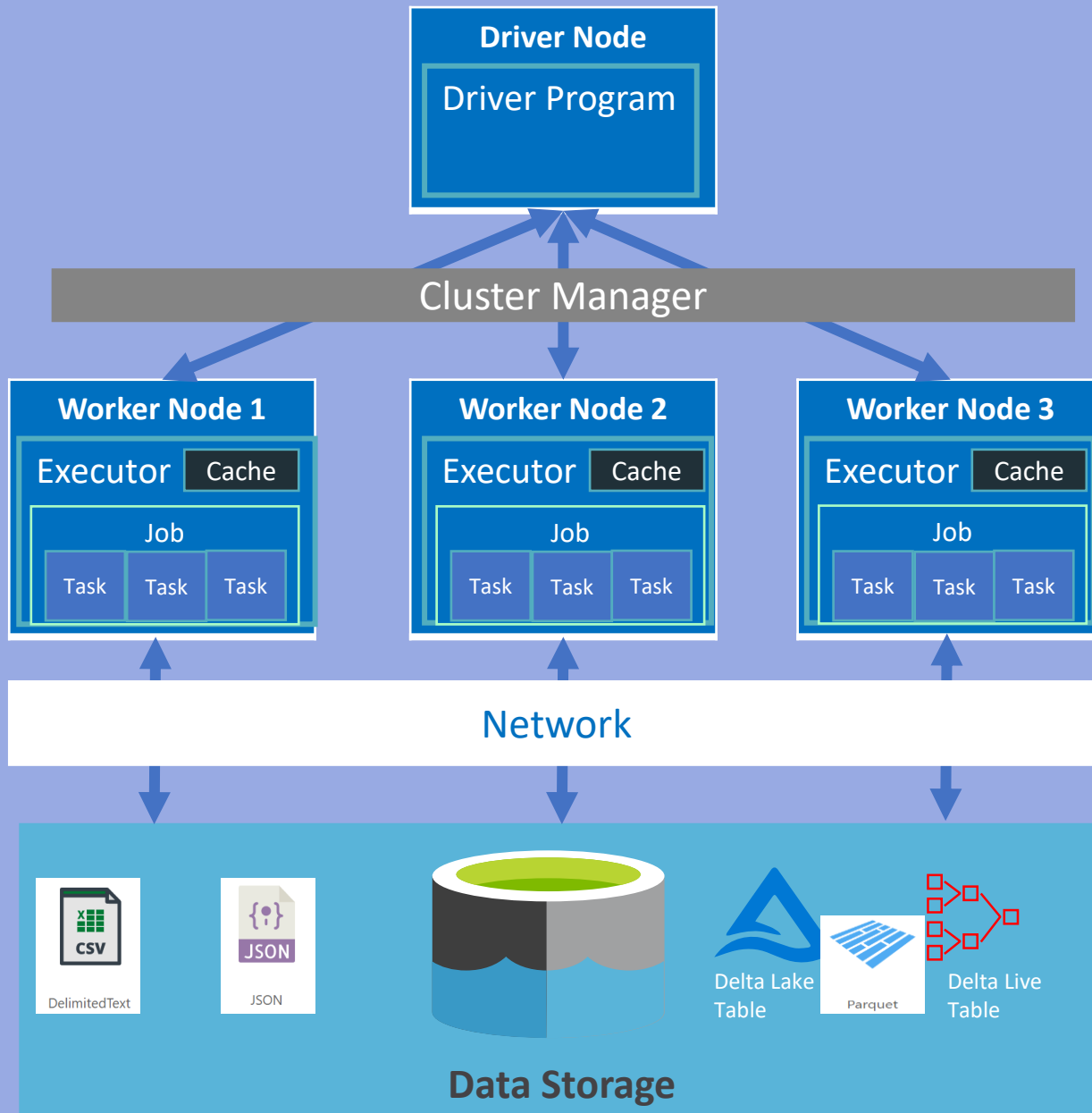
We can Cache the table in memory to find out High Compression Ratio
`spark.table("pricing_analytics.gold.reporting_fact_daily_pricing_gold").cache().count()`

Refer Storage tab in Spark UI to find total Size of Table in Memory

Change Size Of the Data(Default 128 MB) processed by each Shuffle Partition used by AQE to determine number of Shuffle Partitions Using Property set
`spark.databricks.adaptive.autoOptimizeShuffle.preshufflePartitionSizeInBytes = 167772`

Reduce this size and rerun the query to see no Data Spilling , if still exists need to manually tune the Shuffle partitions

Performance Tuning – Data Spilling(Writing to Disk) Issue Contd



Rule Of Thumb for Shuffle Partitions is
2 or 3 times of total number of total worker CPU cores

For manual tuning of Shuffle Partitions need Shuffle Read Size/Records and can be identified from Spark UI

Shuffle Read Size/Records – 100 GB (Ideally 128 MB to 200 MB)

shuffle partitions factor = $\text{ceiling}(\text{Total Size of Shuffle} / \text{Optimal size of data to be processed per task}(128 \text{ MB}) / \text{total worker cores in cluster}(12))$

`select ceiling(10000 / 128) / 12 )`

7

Total number of shuffle partitions = shuffle partitions factor x total worker cores in cluster

84

Calculate the Shuffle Partition for biggest Query in a Notebook and reuse it for all other Queries